



EAST WEST INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING



LABORATORY MANUAL

OBJECT ORIENTED PROGRAMMING WITH JAVA LABORATORY (BCS306A)

III SEMESTER



Prepared By:

Mrs.Anjana.H.S
Assistant Professor,ISE

Mrs K Padmaja
Assistant Professor,ISE

APPROVED BY

Dr.Suresh M.B
Professor & Head,
Dept. of ISE, EWIT

Dr. K. Channakeshavalu
Principal & Director
EWIT, Bangalore.

Name:

USN: *Branch:*

Section: *Batch:*

PROGRAMMING EXPERIMENTS

1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).
2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations.
3. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.
4. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows:
 - Two instance variables x (int) and y (int).
 - A default (or "no-arg") constructor that construct a point at the default location of (0, 0).
 - A overloaded constructor that constructs a point with the given x and y coordinates.
 - A method setXY() to set both x and y.
 - A method getXY() which returns the x and y in a 2-element int array.
 - A toString() method that returns a string description of the instance in the format "(x, y)".
 - A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates
 - An overloaded distance(MyPoint another) that returns the distance from this point to the given MyPoint instance (called another)
 - Another overloaded distance() method that returns the distance from this point to the origin (0,0)Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class.
5. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.
6. Develop a JAVA program to create an abstract class Shape with abstract methods calculateArea() and calculatePerimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape.
7. Develop a JAVA program to create an interface Resizable with methods resizeWidth(int width) and resizeHeight(int height) that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods
8. Develop a JAVA program to create an outer class with a function display. Create another class inside the outer class named inner with a function called display and call the two functions in the main class.
9. Develop a JAVA program to raise a custom exception (user defined exception) for DivisionByZero using try, catch, throw and finally.
10. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.
11. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds).
12. Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

1. **Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).**

```
import java.util.Scanner;
public class Add_Matrix
{
    public static void main(String[] args)
    {
        int p, q, m, n;
        Scanner s = new Scanner(System.in);
        System.out.print("Enter number of rows in first matrix:");
        p = s.nextInt();
        System.out.print("Enter number of columns in first matrix:");
        q = s.nextInt();
        System.out.print("Enter number of rows in second matrix:");
        m = s.nextInt();
        System.out.print("Enter number of columns in second matrix:");
        n = s.nextInt();
        if (p == m && q == n)
        {
            int a[][] = new int[p][q];
            int b[][] = new int[m][n];
            int c[][] = new int[m][n];
            System.out.println("Enter all the elements of first matrix:");
            for (int i = 0; i < p; i++)
            {
                for (int j = 0; j < q; j++)
                {
                    a[i][j] = s.nextInt();
                }
            }
            System.out.println("Enter all the elements of second matrix:");
            for (int i = 0; i < m; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    b[i][j] = s.nextInt();
                }
            }
            System.out.println("First Matrix:");
            for (int i = 0; i < p; i++)
            {
                for (int j = 0; j < q; j++)
                {
                    System.out.print(a[i][j]+" ")
                }
                System.out.println("");
            }
            System.out.println("Second Matrix:");
            for (int i = 0; i < m; i++)
            {

```

```

        for (int j = 0; j < n; j++)
        {
            System.out.print(b[i][j]+" ");
        }
        System.out.println("");
    }
    for (int i = 0; i < p; i++)
    {
        for (int j = 0; j < n; j++)
        {
            for (int k = 0; k < q; k++)
            {
                c[i][j] = a[i][j] + b[i][j];
            }
        }
    }
    System.out.println("Matrix after addition:");
    for (int i = 0; i < p; i++)
    {
        for (int j = 0; j < n; j++)
        {
            System.out.print(c[i][j]+" ");
        }
        System.out.println("");
    }
}
else
{
    System.out.println("Addition would not be possible");
}
}
}

```

OUTPUT

Enter number of rows in first matrix 2
 Enter number of coloumns in first matrix 2
 Enter number of rows in second matrix 2
 Enter number of coloumns in second matrix 2
 Enter all the elements of first matrix:
 1
 2
 3
 4
 5
 Enter all the elements of second matrix:
 1
 2
 3
 4

5

First Matrix

1 2

3 4

Second Matrix

1 2

2 4

Matrix after addition

3 4

6 8

2. **Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations.**

```
import java.io.*;
import java.util.Scanner;
public class stack
{ static int ch;
int element, maxsize, top;
int[] st;
public stack()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter stack size");
maxsize = sc.nextInt();
st = new int[maxsize];
top = -1;
}
public void push(int element)
{
st[++top] = element;
}
}
public int pop()
{
if (top == -1)
{
System.out.println("UnderFlow");
return (-1);
}
return (st[top--]);
}
public void display(int[] st, int max_size)
{
int i;
System.out.println("Stack Elements:");
for (i = 0; i <= max_size; i++)
System.out.println(st[i]);
}
}
public class myStack {
static int ch;
public static void main(String[] args)
{
stack obj = new stack();
while (true)
{
System.out.println("\nEnter your choice\n1.PUSH\n2.POP\n3.Display\n4..EXIT");
Scanner integer = new Scanner(System.in);
ch = integer.nextInt();
switch (ch)
{
case 1: System.out.println("Enter Element");
obj.element = integer.nextInt();
```

```
obj.push(obj.element);
break;
case 2: System.out.printf("Poped element is %d", obj.pop());
break;
case 3: obj.display(obj.st, obj.top);
break;
case 4: System.exit(0);
default: System.out.println("Wrong option"); } } }
```

OUTPUT

```
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
1
Enter Element
11
Inserting 11
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
3
11,
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
1
Enter Element 22
Inserting 22
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
233
Wrong option
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
1
Enter Element 44
Inserting 44
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
1
Enter Element
55
Inserting 55
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
1
Enter Element
66
Inserting 66
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
2
Poped element is ,66
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
3
11, 22, 44, 55,
Enter your choice 1.PUSH 2.POP 3.Display 4..EXIT
4
```

3. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.

```
public class Employee {
    private String name;
    private String jobTitle;
    private double salary;

    public Employee(String name, String jobTitle, double salary) {
        this.name = name;
        this.jobTitle = jobTitle;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public String getJobTitle() {
        return jobTitle;
    }

    public void setJobTitle(String jobTitle) {
        this.jobTitle = jobTitle;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public void raiseSalary(double percentage) {
        salary = salary + salary * percentage / 100;
    }

    public void printEmployeeDetails() {
        System.out.println("Name: " + name);
        System.out.println("Job Title: " + jobTitle);
        System.out.println("Salary: " + salary);
    }
}

public class Main {
    public static void main(String[] args) {
```

```
Employee employee1 = new Employee("Franziska Waltraud", "HR Manager", 40000);
Employee employee2 = new Employee("Hubertus Andrea", "Software Engineer", 60000);
System.out.println("\nEmployee Details:");
employee1.printEmployeeDetails();
employee2.printEmployeeDetails();

employee1.raiseSalary(8);
employee2.raiseSalary(12);

System.out.println("\nAfter raising salary:");
System.out.println("\n8% for 'Franziska Waltraud':");
employee1.printEmployeeDetails();
System.out.println("\n12% for 'Hubertus Andrea':");
employee2.printEmployeeDetails();
}
}
```

OUTPUT

Employee Details:
Name: Franziska Waltraud
Job Title: HR Manager
Salary: 40000.0
Name: Hubertus Andrea
Job Title: Software Engineer
Salary: 60000.0

After raising salary:

8% for 'Franziska Waltraud':
Name: Franziska Waltraud
Job Title: HR Manager
Salary: 43200.0

12% for 'Hubertus Andrea':
Name: Hubertus Andrea
Job Title: Software Engineer
Salary: 67200.0

4. A class called **MyPoint**, which models a 2D point with **x** and **y** coordinates, is designed as follows:
- Two instance variables **x** (int) and **y** (int)
 - A default (or "no-arg") constructor that constructs a point at the default location of (0, 0).
 - A overloaded constructor that constructs a point with the given **x** and **y** coordinates.
 - A method **setXY()** to set both **x** and **y**.
 - A method **getXY()** which returns the **x** and **y** in a 2-element int array.
 - A **toString()** method that returns a string description of the instance in the format "(x, y)".
 - A method called **distance(int x, int y)** that returns the distance from this point to another point at the given (x, y) coordinates
 - An overloaded **distance(MyPoint another)** that returns the distance from this point to the given **MyPoint** instance (called **another**)
 - Another overloaded **distance()** method that returns the distance from this point to the origin (0,0)
- Develop the code for the class **MyPoint**. Also develop a JAVA program (called **TestMyPoint**) to test all the methods defined in the class.

```
class MyPoint
{
    private int x;
    private int y; // Default constructor
    public MyPoint()
    {
        x = 0; y = 0;
    } // Overloaded constructor
    public MyPoint(int x, int y)
    {
        this.x = x;
        this.y = y;
    } // Setter method to set both x and y
    public void setXY(int x, int y)
    {
        this.x = x;
        this.y = y;
    } // Getter method to return x and y in a 2-element

    int array public int[] getXY()
    {
        int[] coordinates = {x, y};
        return coordinates;
    } // toString method to return a string description of the point
    @Override
    public String toString()
    {
        return "(" + x + ", " + y + ")";
    } // Method to calculate distance to another point (x, y)

    public double distance(int x, int y)
    {
        int dx = this.x - x; int dy = this.y - y;
        return Math.sqrt(dx * dx + dy * dy);
    } // Overloaded method to calculate distance to another
    MyPoint instance public double distance(MyPoint another)
```

```

    {
    int dx = this.x - another.x;
    int dy = this.y - another.y;
    return Math.sqrt(dx * dx + dy * dy);
    } // Method to calculate distance to the origin (0, 0)

    public double distance()
    {
    return Math.sqrt(x * x + y * y);
    }
    }
    public class TestMyPoint
    {
    public static void main(String[] args)
    { // Create two MyPoint instances
    MyPoint point1 = new MyPoint(1, 2);
    MyPoint point2 = new MyPoint(4, 6); // Test setXY and getXY methods
    point1.setXY(3, 4);
    int[] coordinates = point1.getXY();
    System.out.println("Coordinates of point1: (" + coordinates[0] + ", " + coordinates[1] + ")"); // Test
    toString method
    System.out.println("Point1: " + point1.toString());
    System.out.println("Point2: " + point2.toString()); // Test distance methods
    System.out.println("Distance between point1 and point2: " + point1.distance(point2));
    System.out.println("Distance from point1 to (0, 0): " + point1.distance());
    System.out.println("Distance from point2 to (0, 0): " + point2.distance(0, 0)); } }

```

OUTPUT

```

Coordinates of point1: (3, 4)
Point1: (3, 4)
Point2: (4, 6)
Distance between point1 and point2: 2.23606797749979
Distance from point1 to (0, 0): 5.0
Distance from point2 to (0, 0): 7.211102550927978

```

5. **Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.**

```
class shape{
void draw()
{
    System.out.println("Drawing Shape");
}
void erase()
{
    System.out.println("erasing Shape");
}
}
class Circle extends shape{
void draw()
{
    System.out.println("Drawing Circle");
}
void erase()
{
    System.out.println("erasing Circle");
}
}
class Triangle extends shape{
void draw()
{
    System.out.println("Drawing Triangle");
}
void erase()
{
    System.out.println("erasing Triangle");
}
}
class Square extends shape{
void draw()
{
    System.out.println("Drawing Square");
}
void erase()
{
    System.out.println("erasing Square");
}
}
class method{
public static void main(String args[]){
    shape c=new Circle();
    shape t=new Triangle();
}
```

```
shape s=new Square();  
c.draw();  
c.erase();  
t.draw();  
t.erase();  
s.draw();  
s.erase();  
}  
}
```

OUTPUT

Drawing Circle
erasing Circle
Drawing Triangle
erasing Triangle
Drawing Square
erasing Square

6. **Develop a JAVA program to create an abstract class Shape with abstract methods calculateArea() and calculatePerimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape**

```
abstract class Shape {  
    abstract double calculateArea();  
    abstract double calculatePerimeter();  
}
```

```
// Circle.java  
// Subclass Circle  
class Circle extends Shape {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    @Override  
    double calculateArea() {  
        return Math.PI * radius * radius;  
    }  
  
    @Override  
    double calculatePerimeter() {  
        return 2 * Math.PI * radius;  
    }  
}
```

```
// Triangle.java  
// Subclass Triangle  
class Triangle extends Shape {  
    private double side1;  
    private double side2;  
    private double side3;  
  
    public Triangle(double side1, double side2, double side3) {  
        this.side1 = side1;  
        this.side2 = side2;  
        this.side3 = side3;  
    }  
  
    @Override  
    double calculateArea() {  
        double s = (side1 + side2 + side3) / 2; // Semi-perimeter  
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));  
    }  
}
```

```

@Override
double calculatePerimeter() {
    return side1 + side2 + side3;
}
}
// Main.java
// Subclass Main

class Main {
public static void main(String[] args) {
    double r = 4.0;
    Circle circle = new Circle(r);
    double ts1 = 3.0, ts2 = 4.0, ts3 = 5.0;
    Triangle triangle = new Triangle(ts1, ts2, ts3);
    System.out.println("Radius of the Circle"+r);
    System.out.println("Area of the Circle: " + circle.calculateArea());
    System.out.println("Perimeter of the Circle: " + circle.calculatePerimeter());
    System.out.println("\nSides of the Traiangel are: "+ts1+', '+ts2+', '+ts3);
    System.out.println("Area of the Triangle: " + triangle.calculateArea());
    System.out.println("Perimeter of the Triangle: " + triangle.calculatePerimeter());
}
}

```

OUTPUT

Radius of the circle: 4.0
 Area of the circle : 50.265
 Perimeter of the circle: 25.13

Sides of Triangle are: 3.0 4.0 5.0
 Area of the Triangle: 6.0
 Perimeter of the triangle: 12.0

7. Develop a JAVA program to create an interface Resizable with methods `resizeWidth(int width)` and `resizeHeight(int height)` that allow an object to be resized. Create a class `Rectangle` that implements the `Resizable` interface and implements the resize methods

```
interface Resizable {
    void resizeWidth(int width);
    void resizeHeight(int height);
}

//Rectangle.java
class Rectangle implements Resizable {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public void resizeWidth(int width) {
        this.width = width;
    }

    public void resizeHeight(int height) {
        this.height = height;
    }

    public void printSize() {
        System.out.println("Width: " + width + ", Height: " + height);
    }
}

//Main.java
public class Main {
    public static void main(String[] args) {
        Rectangle rectangle = new Rectangle(100, 150);
        rectangle.printSize();

        rectangle.resizeWidth(150);
        rectangle.resizeHeight(200);
        rectangle.printSize();
    }
}
```

OUTPUT

```
Width: 100, Height: 150
Width: 150, Height: 200
```

8. Develop a JAVA program to create an outer class with a function display. Create another class inside the outer class named inner with a function called display and call the two functions in the main class.

```
class Outer{
String so = ("This is Outer Class");
void display()
{
System.out.println(so);
}
void test(){
Inner inner = new Inner();
inner.display();
}
//this is an inner class
class Inner{
String si =("This is inner Class");
void display(){
System.out.println(si);
}
}
}
class InnerClassDemo{
public static void main(String args[]){
Outer outer = new Outer();
outer.display();
outer.test();
}
}
```

OUTPUT

This is Outer Class
This is inner Class

9. Develop a JAVA program to raise a custom exception (user defined exception) for DivisionByZero using try, catch, throw and finally.

```
import java.util.*;
public class TryP
{
    int c;
    void div(int a,int b)
    {
        try {
            c=a/b;
            System.out.println("Result="+c);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Cannot divide by zero");
        }
    }
    public static void main(String args[])
    {
        TryP obj=new TryP();
        Scanner in=new Scanner(System.in);
        System.out.println("Enter the values of a and b");
        int no1=in.nextInt();
        int no2=in.nextInt();
        obj.div(no1,no2);
    }
}
```

OUTPUT

```
Enter the values of a and b
10
2
Result=5
Enter the values of a and b
4
0
Cannot divide by zero
```

10. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.

Follow these steps

1. Create a directory structure to organize your code. You should have a directory named "mypack" to represent the package and a separate directory for your main class.

2. Inside the "mypack" directory, create a Java file with a class that you want to include in the package. For example, let's create a class called MyClass in a file named MyClass.java

// File: mypack/MyClass.java

```
package mypack;
public class MyClass
{
    public void display()
    {
        System.out.println("This is a method from the 'mypack' package.");
    }
}
```

Create a separate java file for main class that will import and use the Class from the “mypack” package

Now, create a separate Java file for your main class that will import and use the class from

// File: MainClass.java

```
import mypack.MyClass;
public class MainClass
{
    public static void main(String[] args)
    {
        MyClass myObj = new MyClass();
        myObj.display();
    }
}
```

OUTPUT

This is a method from the 'mypack' package.

11. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds)

```
class IntThread implements Runnable
{
    Thread t;
    IntThread()
    {
        t = new Thread ( this, "Test thread");
        System.out.println (" Child thread :" + t);
        t.start();
    }
    public void run()
    {
        try {
            for (int i= 5; i0; j--)
            {
                System.out.println ("Main thread :" + j);
                Thread.sleep (1000);
            }
        } catch (InterruptedException e) { }
        System.out.println ( "Main thread exiting ...");
    }
}
```

OUTPUT

Thread Thread 2 is running.
Thread Thread 1 is running.
Thread Thread 1 has completed.
Thread Thread 2 has completed.

12. Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

```
class MyThread extends Thread
{
    MyThread()
    {
        super ("Using Thread class");
        System.out.println ("Child thread:" + this);
        start();
    }
    public void run()
    {
        try
        {
            for ( int i =5; i > 0; i--)
            {
                System.out.println ("Child thread" + i);
                Thread.sleep (500);
            }
        } catch (InterruptedException e) { }
        System.out.println ("exiting child thread ...");
    }
}
class TestMyThread
{
    public static void main(String args[])
    {
        new MyThread();
        try {
            for ( int k = 5; k > 0; k--)
            {
                System.out.println ("Running main thread :" + k);
                Thread.sleep(1000);
            }
        } catch (InterruptedException e) { }
        System.out.println ("Exiting main thread . . .");
    }
}
```

OUTPUT

```
Child Thread: ChildThread - Count: 1
Main Thread - Count: 1
Main Thread - Count: 2
Child Thread: ChildThread - Count: 2
Main Thread - Count: 3
Child Thread: ChildThread - Count: 3
Main Thread - Count: 4
Child Thread: ChildThread - Count: 4
Main Thread - Count: 5
Child Thread: ChildThread - Count:5
```